

LARANA, INC.

Smoke Detection Using IOT Sensor Data

AYMAN SOHAIL - 100963747 SHAWNZEY SYED - 100864082
IZZAT IKRAM - 100987847 FATIMAH KAMAL - 100924838

IoT-BASED SMOKE AND FIRE DETECTION

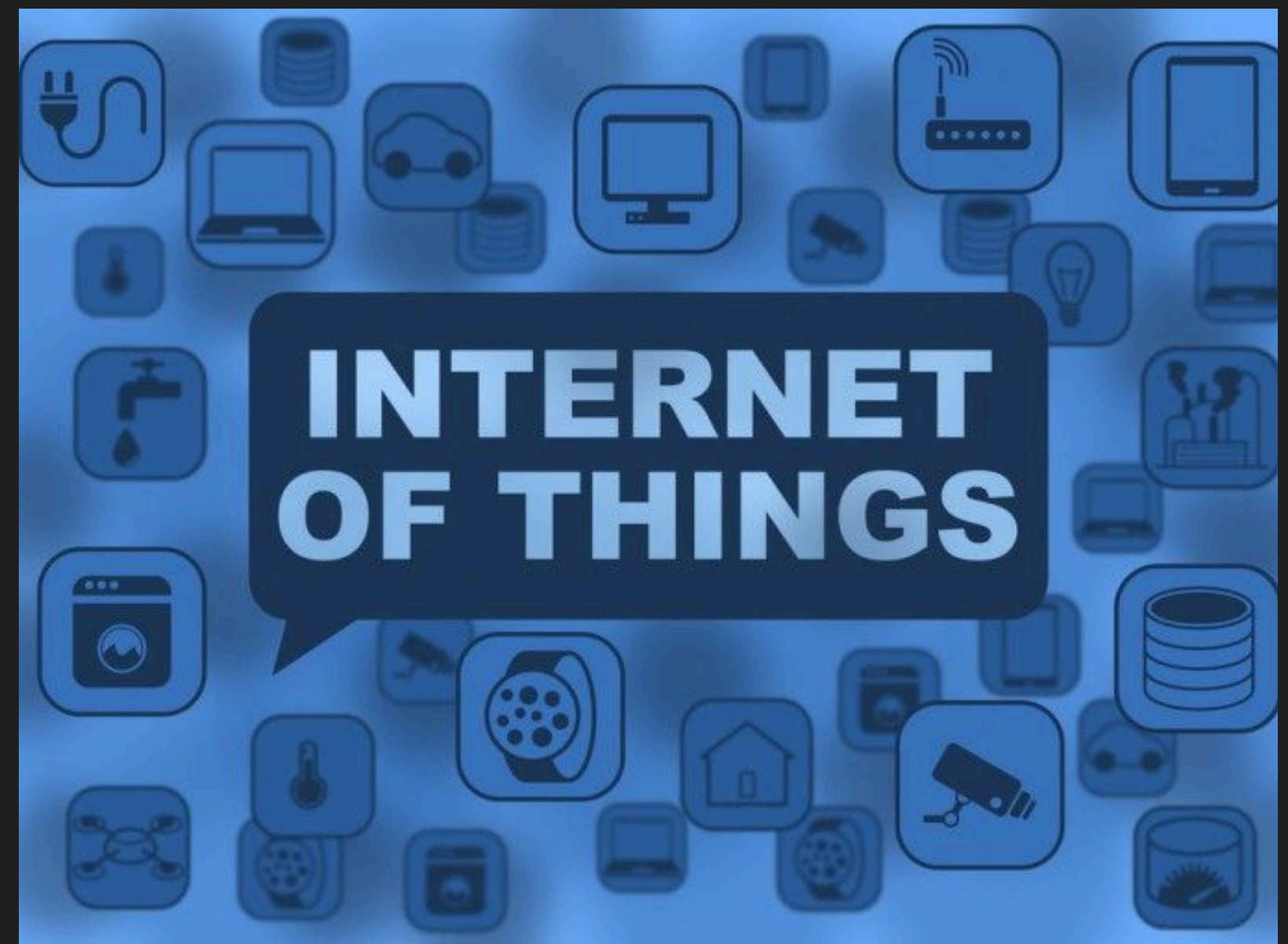
Internet of Things (IoT) enables real-time monitoring
Collect data from the physical world

Used in smart homes, industries, and safety systems

Data-Driven Decision Making
Sensor data allows smarter and faster responses

Enhancing Smoke Detection Systems
Combines multiple sensors for early and accurate fire detection

...





- Traditional systems rely only on smoke, heat, and flame detection (Liu, 2025)
- Limited data inputs reduce detection accuracy
- High risk of false alarms or delayed response
- Lack of environmental context (e.g., gas levels, humidity)
- Need for a more intelligent, data-driven detection system

...

Limitations of Traditional Fire Alarms



Project Objective

Use IoT sensor data to build a machine learning model that can detect fire alarms

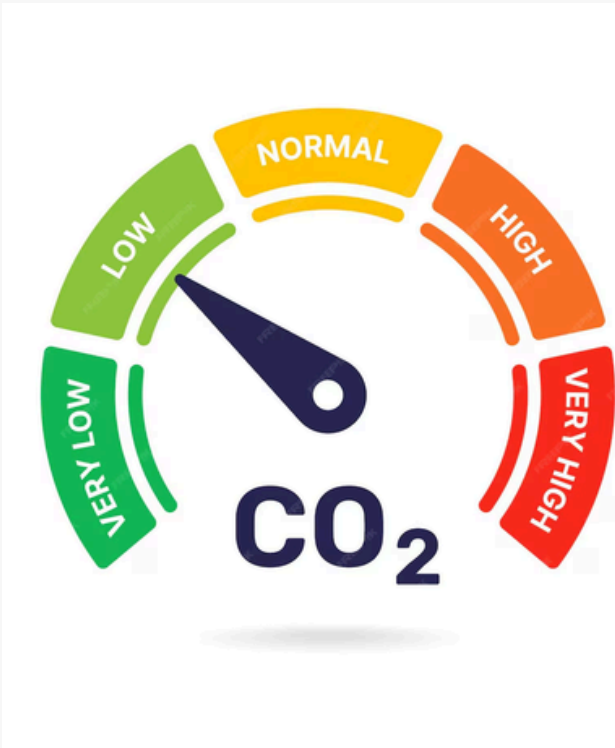
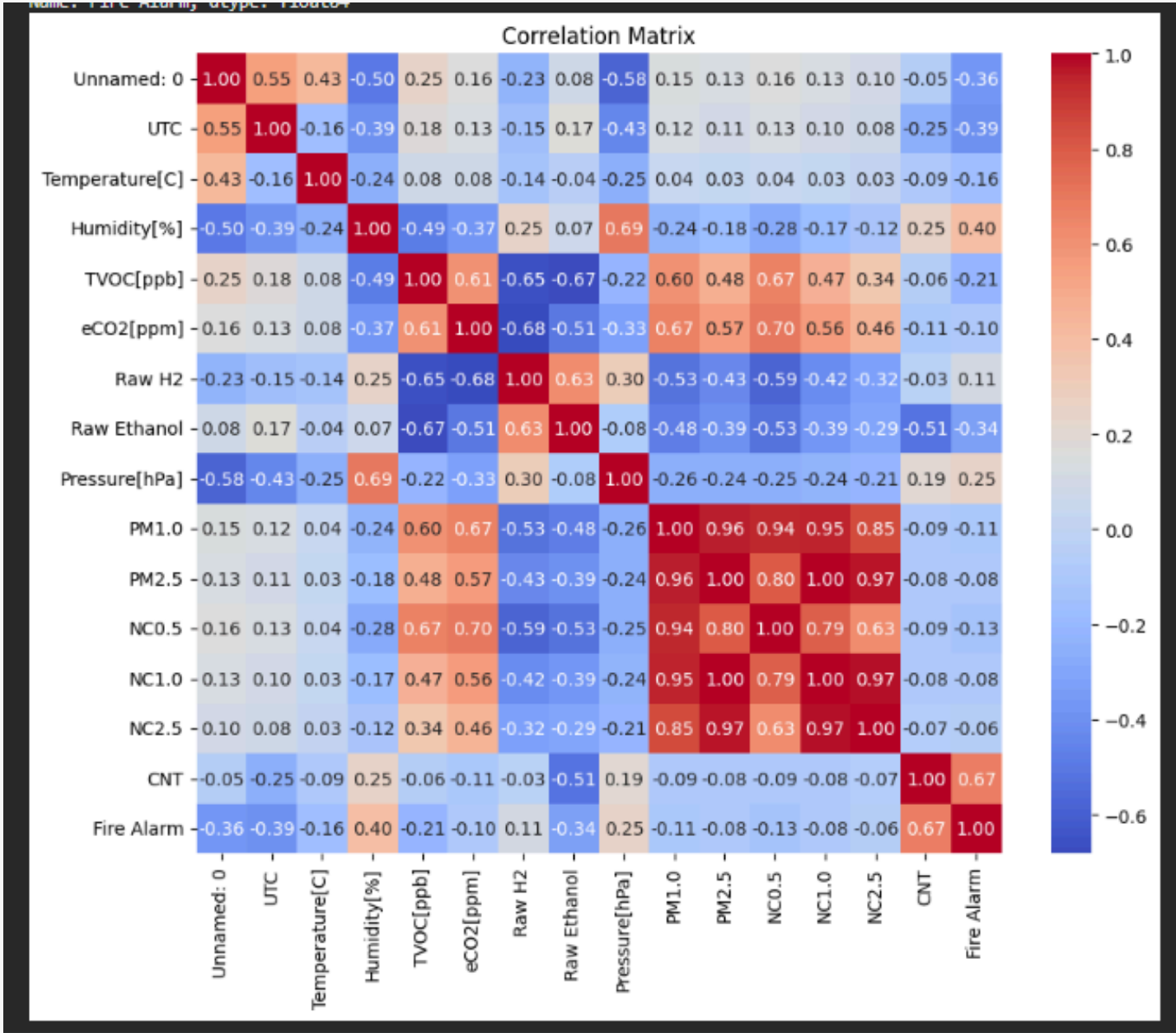
Compare multiple ML models (such as KNN & Random Forest) and Improve detection accuracy compared to traditional threshold-based systems

Analyze environmental sensor inputs: temperature, humidity, gas concentration such as CO, smoke levels) and air pressure

Use accuracy, precision, recall and F1-score to evaluate the model performance and Check feasibility of deployment on IoT edge devices such as Raspberry Pi.

Data Set

- KAGGLE SMOKE DETECTION DATASET WITH 62,630 RECORDS AND 16 FEATURES
- TARGET VARIABLE: FIRE ALARM
(0= NO FIRE, 1= FIRE)
- FEATURES INCLUDE ENVIROMENTAL, GAS AND PARTICLE SENSORS
- WE NOTICED KEY FEAUTURES LIKE CNT, HUMIDITY AND PRESSURE STRONGLY RELATE TO FIRE DETECTION



Methodology

ALGORITHMS USED

K-Nearest Neighbors
(KNN)

Random Forest Classifier

Exploratory Data Analysis (EDA)

```
# Check for missing values (if any)
df.isnull().sum()
```

	0
Unnamed: 0	0
UTC	0
Temperature[C]	0
Humidity[%]	0
TVOC[ppb]	0
eCO2[ppm]	0
Raw H2	0
Raw Ethanol	0
Pressure[hPa]	0
PM1.0	0
PM2.5	0
NC0.5	0
NC1.0	0
NC2.5	0
CNT	0
Fire Alarm	0

dtype: int64

- Printed first 5 rows to view dataset
- No missing values seen
- All values numerical, no conversions needed
- Decided to drop Unnamed and UTC features

```
# Print basic information about the dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 62630 entries, 0 to 62629
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Unnamed: 0             62630 non-null  int64  
1   UTC                    62630 non-null  int64  
2   Temperature[C]         62630 non-null  float64
3   Humidity[%]            62630 non-null  float64
4   TVOC[ppb]              62630 non-null  int64  
5   eCO2[ppm]              62630 non-null  int64  
6   Raw H2                 62630 non-null  int64  
7   Raw Ethanol            62630 non-null  int64  
8   Pressure[hPa]          62630 non-null  float64
9   PM1.0                  62630 non-null  float64
10  PM2.5                  62630 non-null  float64
11  NC0.5                  62630 non-null  float64
12  NC1.0                  62630 non-null  float64
13  NC2.5                  62630 non-null  float64
14  CNT                    62630 non-null  int64  
15  Fire Alarm             62630 non-null  int64  
dtypes: float64(8), int64(8)
memory usage: 7.6 MB
```

```
# Display the first five rows of the DataFrame to understand its structure
df.head()
```

	Unnamed: 0	UTC	Temperature[C]	Humidity[%]	TVOC[ppb]	eCO2[ppm]	Raw H2	Raw Ethanol	Pressure[hPa]	PM1.0	PM2.5	NC0.5	NC1.0	NC2.5
0	0	1654733331	20.000	57.36	0	400	12306	18520	939.735	0.0	0.0	0.0	0.0	0.0
1	1	1654733332	20.015	56.67	0	400	12345	18651	939.744	0.0	0.0	0.0	0.0	0.0
2	2	1654733333	20.029	55.96	0	400	12374	18764	939.738	0.0	0.0	0.0	0.0	0.0
3	3	1654733334	20.044	55.28	0	400	12390	18849	939.736	0.0	0.0	0.0	0.0	0.0
4	4	1654733335	20.059	54.69	0	400	12403	18921	939.744	0.0	0.0	0.0	0.0	0.0

Running KNN & Random Forest Classifier

STEPS

Prepare data for training

Train the model

Make Predictions on test set

Obtain Precision, Recall, Model

Accuracy and F1 Score

Visualize ROC curve, Confusion Matrix
and Sample Predictions

```
from sklearn.metrics import f1_score
from sklearn.metrics import precision_score, recall_score

# Make predictions on the test set
y_pred = model.predict(X_test)

precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
print(f"Precision: {precision}")
print(f"Recall: {recall}")

# Calculate and print the Accuracy to evaluate the model's performance
accuracy = model.score(X_test, y_test)
print(f"Model Accuracy: {accuracy}")

# Calculate and print the F1-score
f1 = f1_score(y_test, y_pred)
print(f"F1 Score: {f1}")
```

Tuning Parameters

```
import time

for k in [3, 5, 7, 9, 11]:
    start = time.time()
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    acc = model.score(X_test, y_test)
    end = time.time()
    print(f"k={k}, Accuracy: {acc}, Runtime: {round(end-start, 4)}s")
```

```
k=3, Accuracy: 0.9996806642184257, Runtime: 0.5763s
k=5, Accuracy: 0.9997604981638193, Runtime: 0.5187s
k=7, Accuracy: 0.9996008302730321, Runtime: 0.5262s
k=9, Accuracy: 0.9997604981638193, Runtime: 0.6118s
k=11, Accuracy: 0.9998403321092129, Runtime: 0.5637s
```

```
for n in [10, 50, 100, 200]:
    start = time.time()
    model = RandomForestClassifier(n_estimators=n)
    model.fit(X_train, y_train)
    acc = model.score(X_test, y_test)
    end = time.time()
    print(f"n={n}, Accuracy: {acc}, Runtime: {round(end-start, 4)}s")
```

```
n=10, Accuracy: 1.0, Runtime: 0.4413s
n=50, Accuracy: 1.0, Runtime: 2.7786s
n=100, Accuracy: 1.0, Runtime: 3.8318s
n=200, Accuracy: 1.0, Runtime: 8.6845s
```

```
# Separate the features (X) and target (y)
X = df.drop(['Fire Alarm', 'UTC', 'Unnamed: 0'], axis=1)
y = df['Fire Alarm'] # Target (fire alarm, 1 or 0)
```

```
# Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Print the shape of the training and testing datasets to verify the split
print(f'Training data shape: {X_train.shape}, Training labels shape: {y_train.shape}')
print(f'Test data shape: {X_test.shape}, Test labels shape: {y_test.shape}')
```

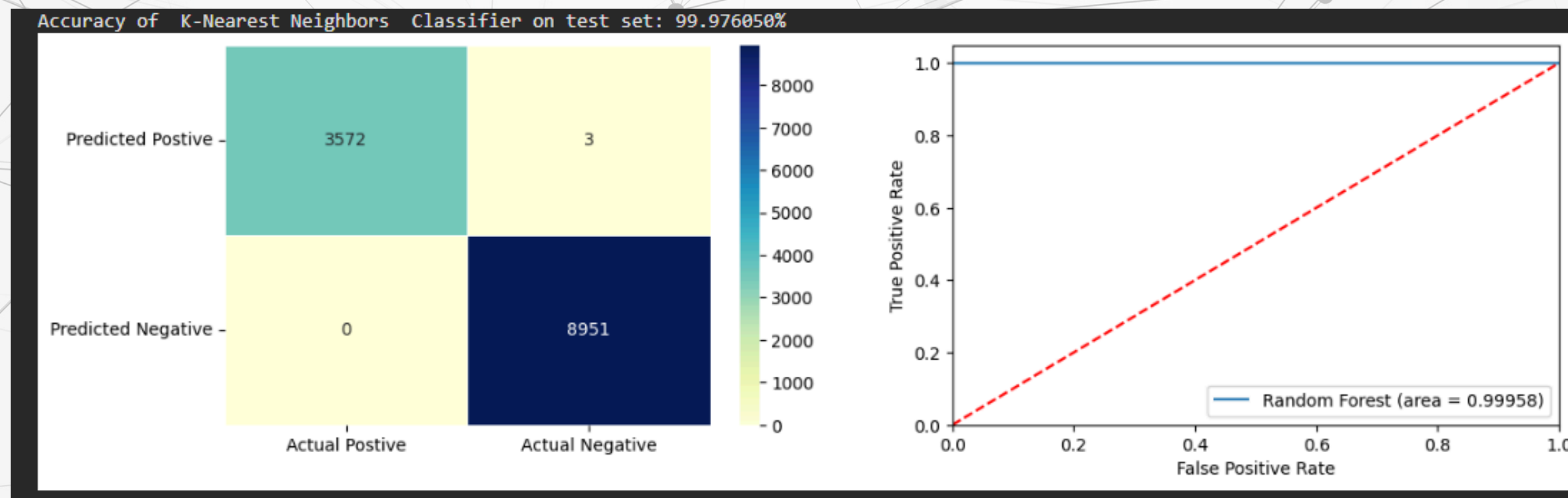
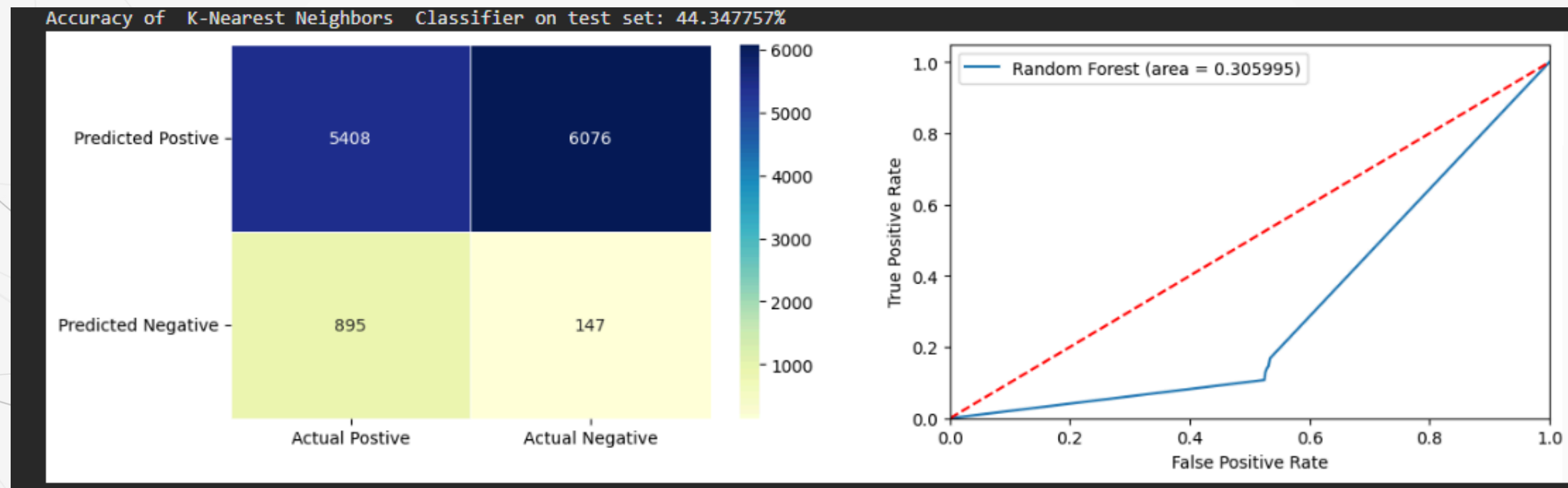
```
# Apply the models to the dataset
def applyModel(X_train, X_test, y_train, y_test, selectedModel, plotFlag=True):
    modelTitle = selectedModel[0]
    absTitle = selectedModel[1]
    model=""

    if modelTitle == "RandomForest": # Random Forest
        model = RandomForestClassifier(n_estimators= 10)

    elif modelTitle == "K-Nearest Neighbors": # K-Nearest Neighbors
        model = KNeighborsClassifier(n_neighbors=11)

    if model != "": # if model is not empty
        model.fit(X_train, y_train) # Train the model
        y_pred, model_score, predict_proba = model.predict(X_test), model.score(X_test, y_test), model.predict_proba(X_test)
        if plotFlag:
            print('Accuracy of ', modelTitle, ' Classifier on test set: {:.6f}%'.format(model_score*100))

    tn, fp, fn, tp = accuracy_vis(X_test, y_test, y_pred, predict_proba, plotFlag) # Call the accuracy_vis function
    result.loc[absTitle] = [modelTitle, tn, fp, fn, tp, round(model_score*100, 6)] # Store the results in the result dataframe
```



KNN

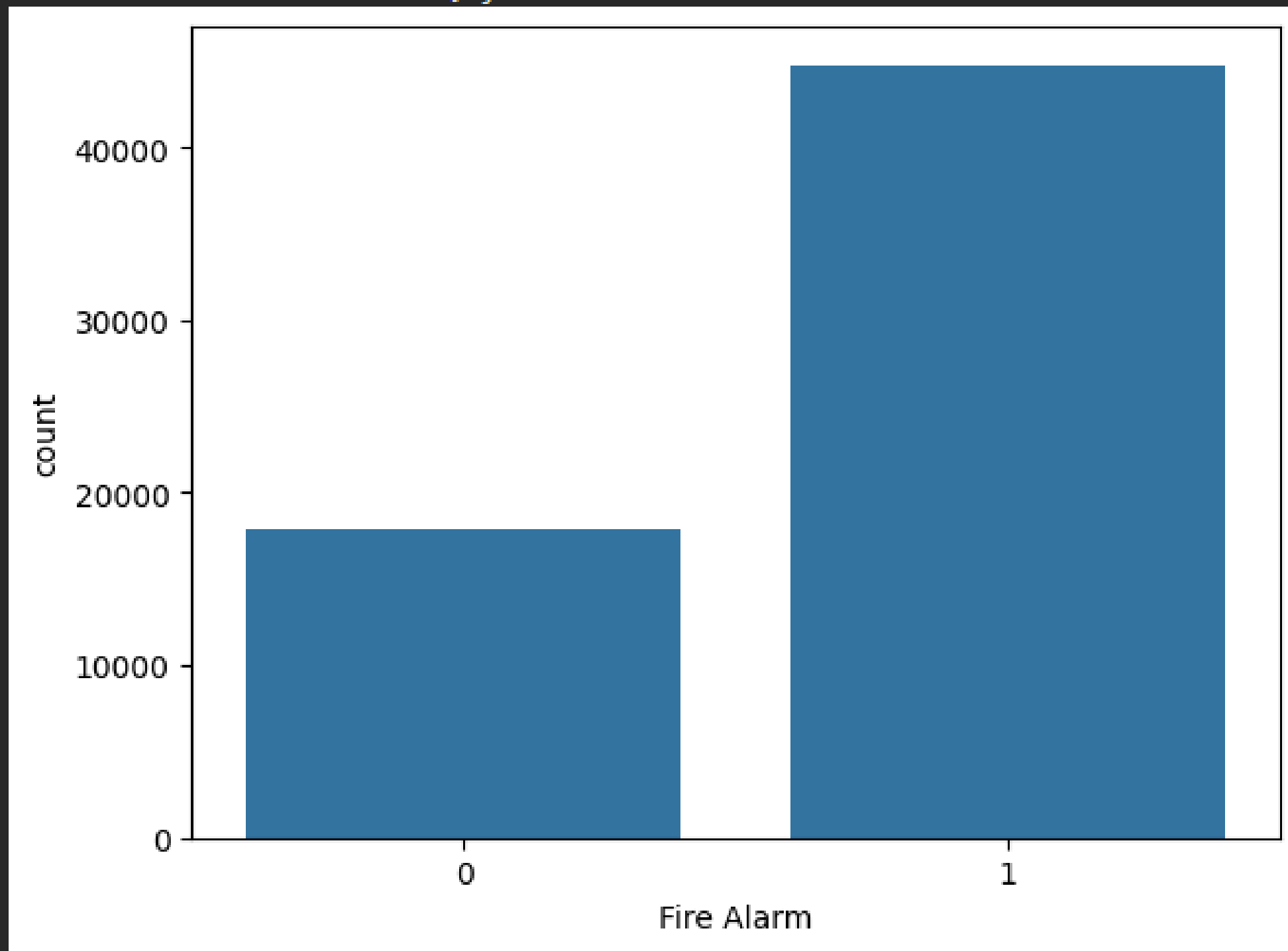
The Reason

...

Fire Alarm		count
1		44757
0		17873

dtype: int64

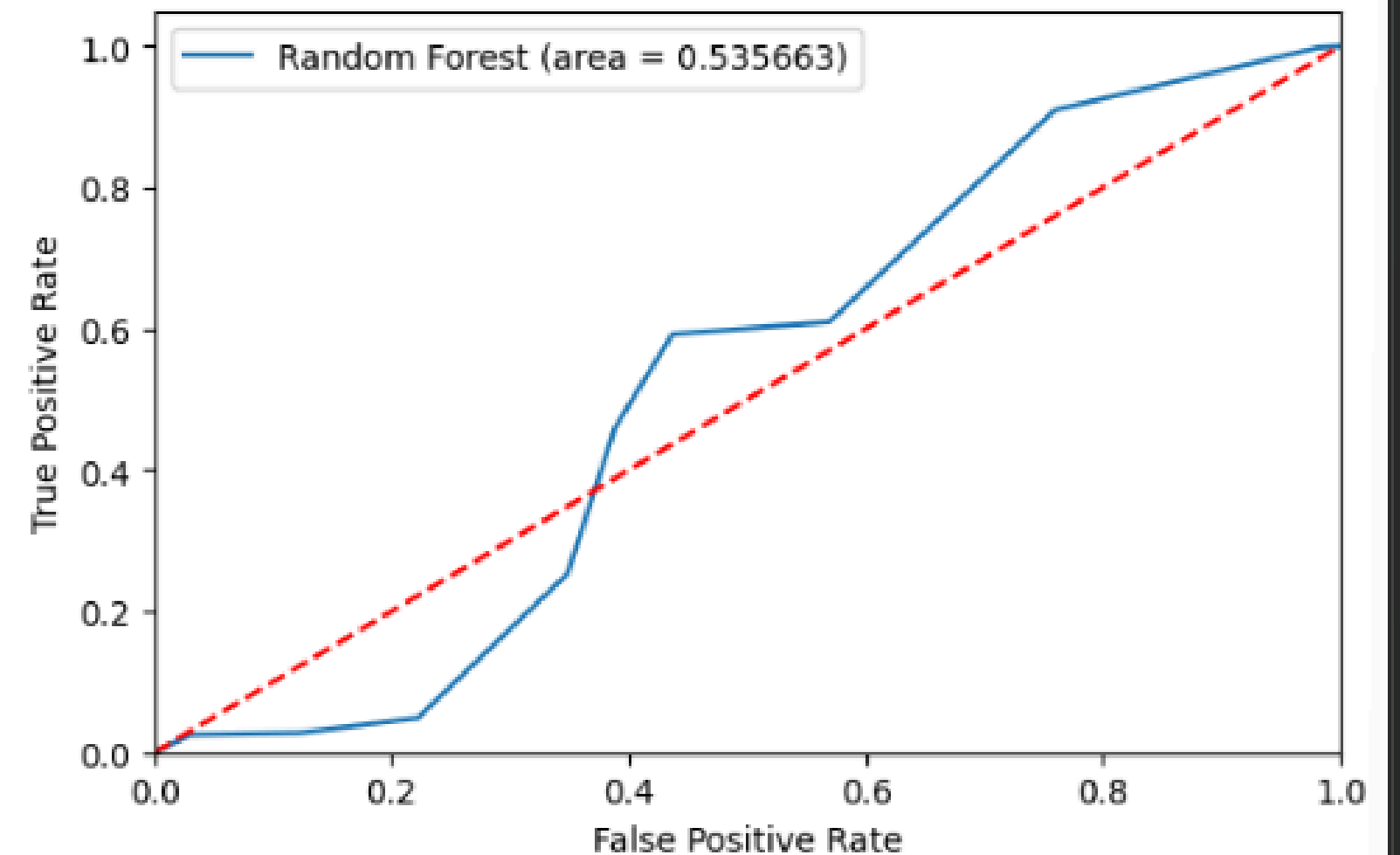
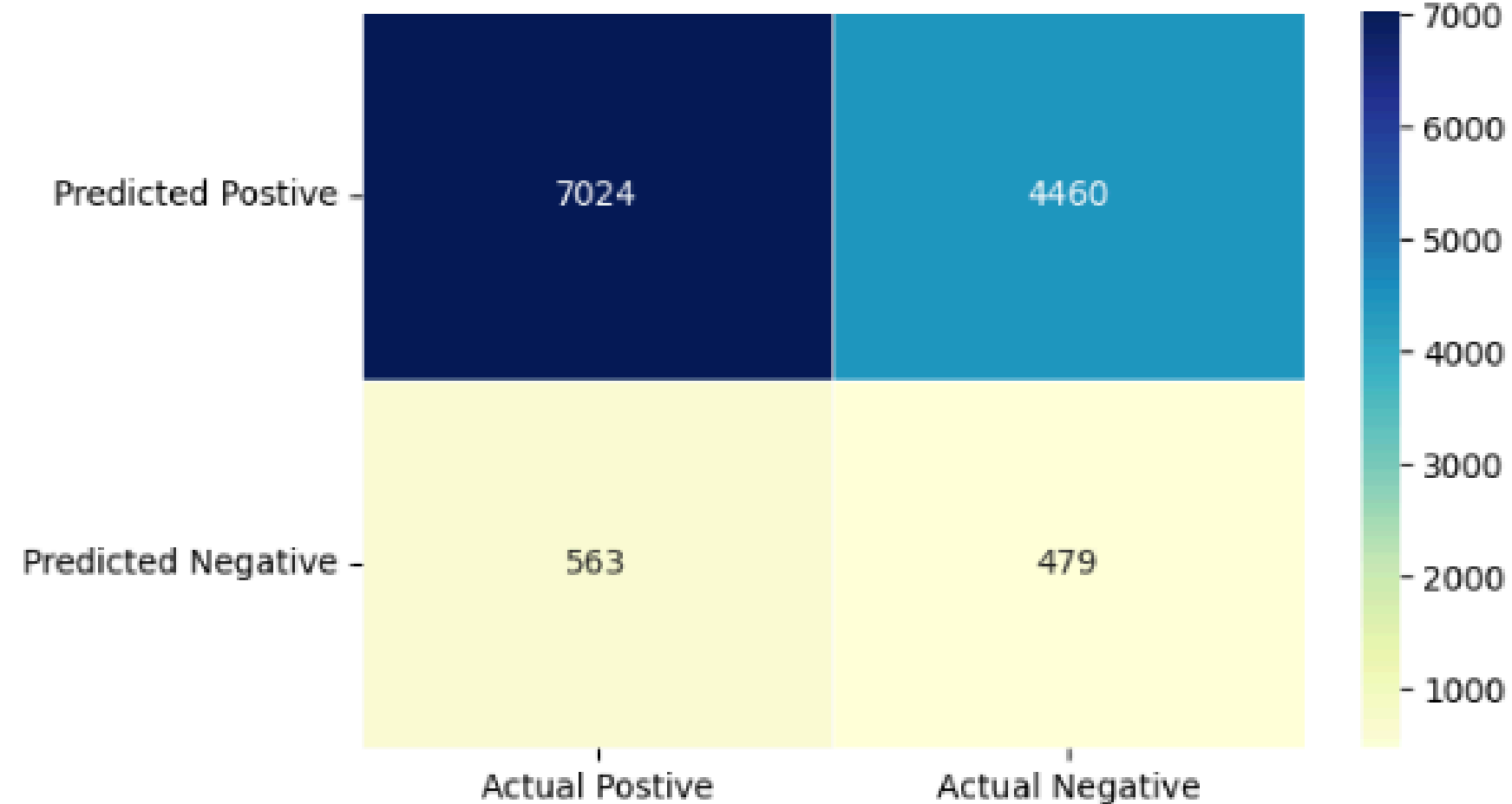
.. <Axes: xlabel='Fire Alarm', ylabel='count'>



BEFORE

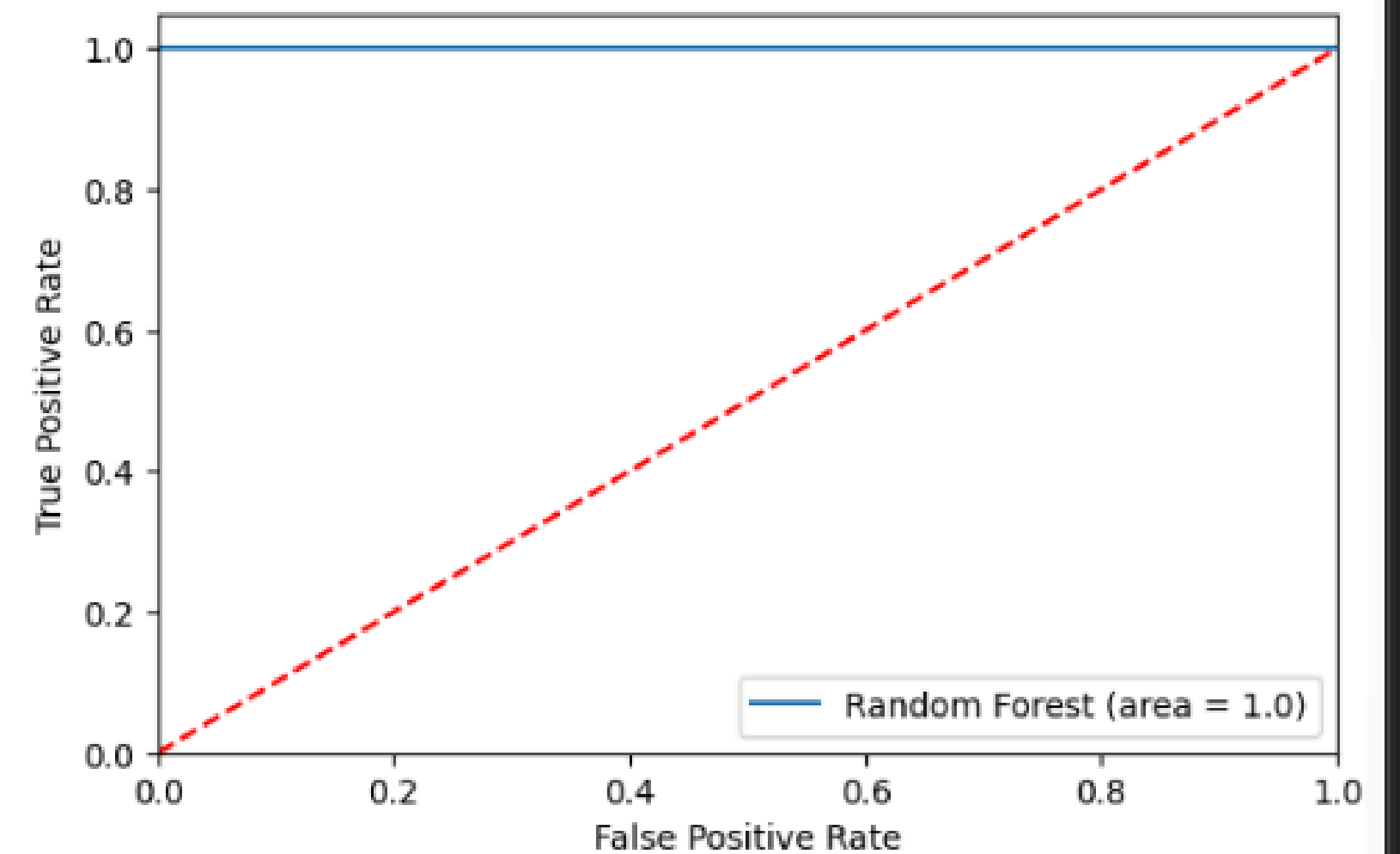
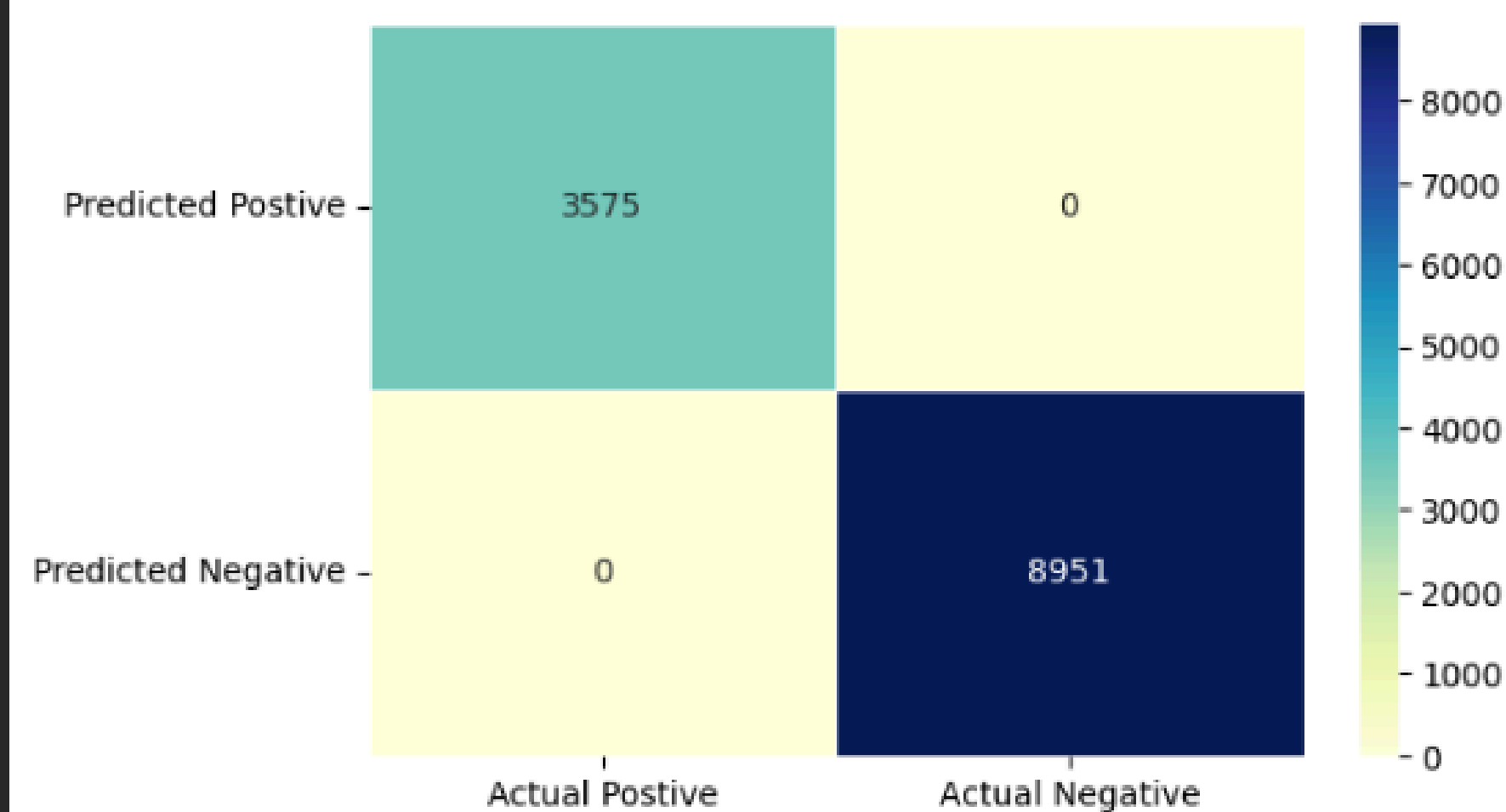
```
... Precision: 0.04414229837958652
Recall: 0.22744721689059502
Model Accuracy: 0.5260258661983075
F1 Score: 0.07393542349087506
```

Accuracy of RandomForest Classifier on test set: 59.899409%



Precision: 1.0
Recall: 1.0
Model Accuracy: 1.0
F1 Score: 1.0

Accuracy of RandomForest Classifier on test set: 100.000000%



AFTER

Comparing Results

Classifier Accuracy Comparison

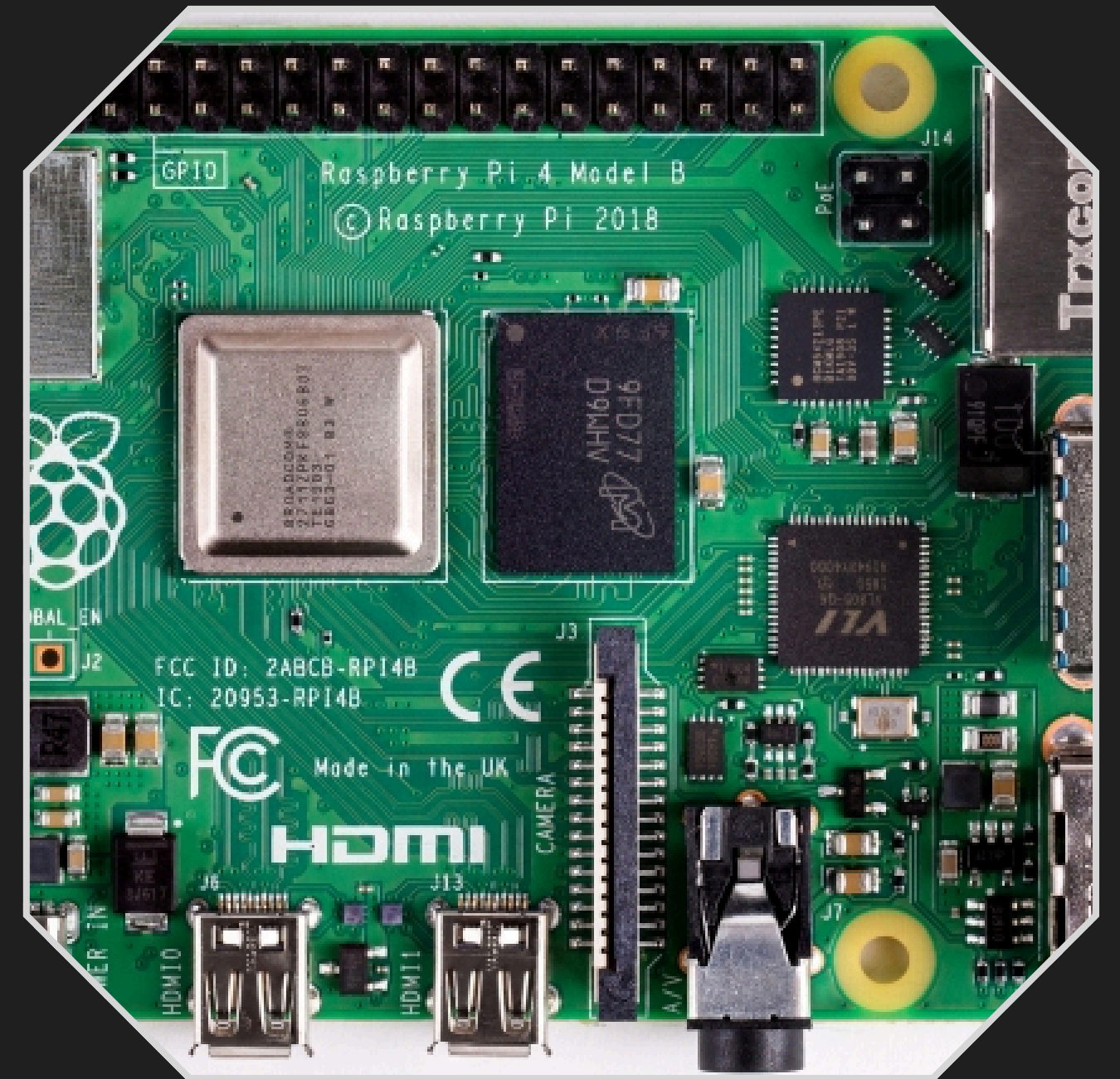
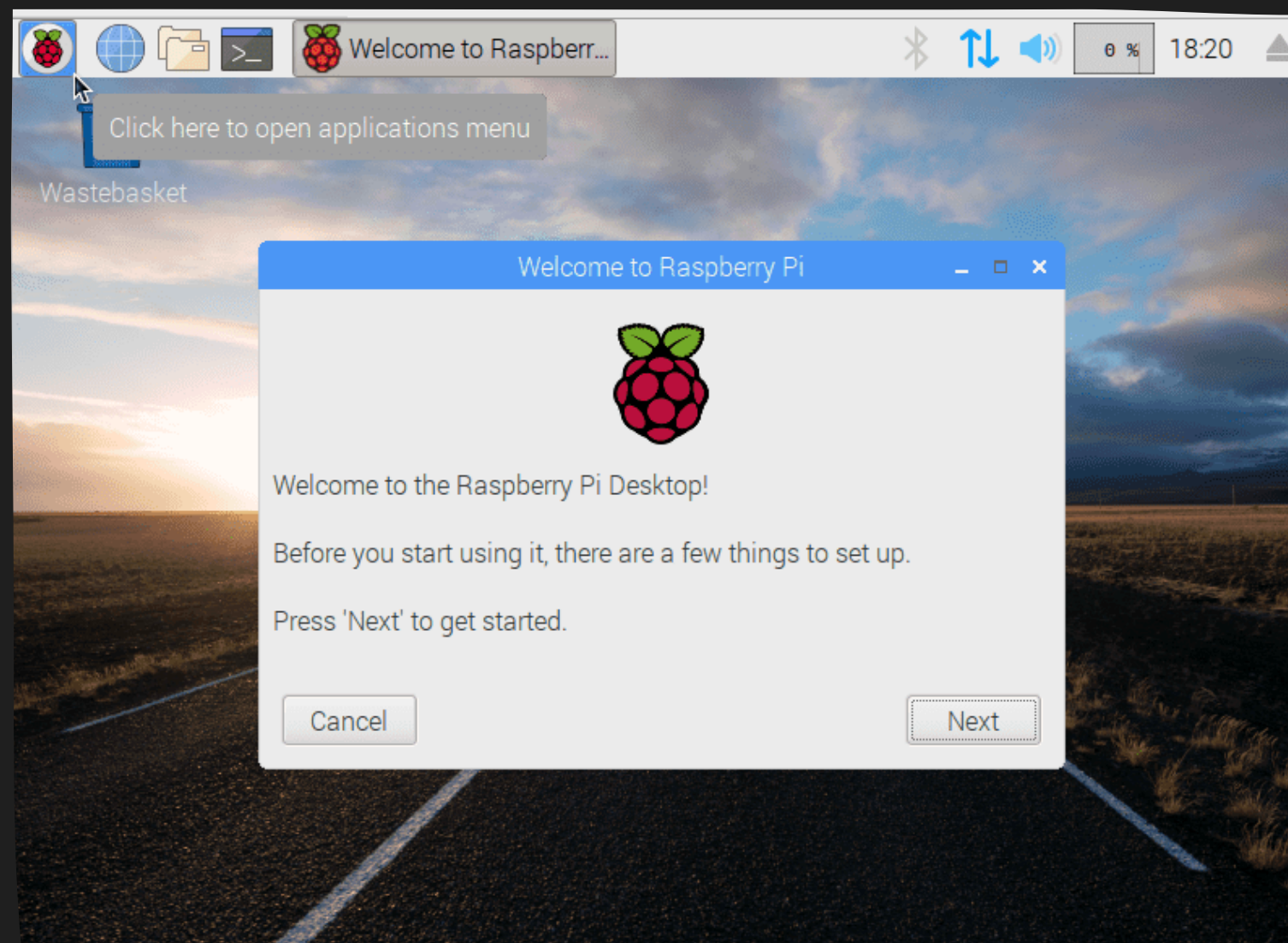
	Classifier	True Negative	False Postive	False Negative	True Positive	Classifier Accuracy
RF	RandomForest	3594	0	0	8932	100.000000
KNN	K-Nearest Neighbors	3592	2	0	8932	99.984033

Conclusion: We built models which can predict fire alarms with an accuracy of 100%, analyzing features more than traditional fire alarms.

However, 100% is a **RED FLAG**, since that is not realistically possible. therefore we must understand that there is scope for refinement of the RF model to ensure accuracy.

Next Steps

Optimize models for running on low memory usage and test our models in a IoT environment: Raspberry Pi





Thank You

REFERENCES

Professor resource:
Tutorial 1 Room Occupancy code

Code sampled from :
<https://www.kaggle.com/datasets/deepcontractor/smoke-detection-dataset>